

NES555 — Lab 2: Passive Reconnaissance & Footprinting

Course: NES555 — Ethical Hacking Lab

Lab environment: Kali Linux VM (attacker) — use only the course lab network and instructor-approved targets.

Lab overview — background and purpose

Passive reconnaissance is the first information-gathering phase of any ethical penetration test. Unlike active scanning, *passive* techniques collect publicly available information (DNS records, certificates, search-engine indexes, public code repositories, Shodan banners, etc.) without sending probes directly to a target host. This reduces risk of detection and legal exposure while building a rich target profile that guides later (authorized) tests.

In this lab you will practice passive OSINT and footprinting using Kali tools plus browser-based Google dorking and Shodan (free account). You will also use **recon-ng** — a powerful framework that automates many passive collection tasks and helps you store and export findings in structured form.

Learning objectives

By the end of this lab you will be able to:

1. Explain the role of passive reconnaissance in a penetration test.
 2. Collect DNS, WHOIS, and certificate-transparency data for a domain.
 3. Use search engines and `theHarvester` to find email addresses and hosts.
 4. Enumerate subdomains using passive tools (`sublist3r`, `amass`) and correlate results.
 5. Perform controlled Google dorking and interpret results responsibly.
 6. Query Shodan (CLI) to find passive banner data for discovered hosts.
 7. Use `recon-ng` to automate passive collection, store results, and export a findings report.
-

Prerequisites (host & VM state)

- Kali VM up and attached to the lab network (host-only or instructor network).
- You are logged into Kali (root or sudo-capable account).

- Tools installed on Kali (most are in default images). If missing, install:
 - `sudo apt update`
 - `sudo apt install -y theharvester sublist3r amass recon-ng shodan jq curl whois dnsutils`
 - Instructor-provided target domain(s) or approved public/demo domains (e.g., `testphp.vulnweb.com`, `demo.testfire.net`, or a local Juice Shop URL).
-

Lab safety & rules (read first)

- **Passive only:** No port scans, no brute force, no exploitation. Do **not** run `nmap`, `masscan`, or other active probes against public targets unless explicitly authorized.
 - Operate only on instructor-approved/demo domains or the local lab domains.
 - Do not download or exfiltrate files found via dorks—record their existence and URL only.
 - Document sources and timestamps for every finding.
-

Tools (quick reference)

- CLI: `whois`, `dig`, `nslookup`, `curl`, `jq`
 - OSINT: `theHarvester`, `sublist3r`, `amass` (passive mode)
 - Automation: `recon-ng` (framework)
 - Web: Browser (Google dorking)
 - Shodan: `shodan` CLI (requires free API key)
-

Tasks

Replace `domain` with the assigned domain (e.g., `testphp.vulnweb.com`) and `YOUR_SHODAN_KEY` with your Shodan API key.

Task 0 — Preparation (5–10 minutes) — background

Before collecting, confirm network access and prepare workspace folders. Keep a terminal open for commands and one browser tab for Google dorks.

Commands

```
mkdir -p ~/nes555/lab2
```

```
cd ~/nes555/lab2
whoami && hostname && date
```

Notes

Record the date/time and the domain(s) you will investigate; this helps reproducibility.

Task 1 — WHOIS & DNS lookups (15 minutes) — background

Why: WHOIS provides registrar/registrant metadata, while DNS queries reveal authoritative name servers, MX records, and basic host-to-IP mapping.

Commands

```
# WHOIS (brief)
whois domain | sed -n '1,40p'

# DNS A record(s)
dig +noall +answer domain

# NS records
dig +short NS domain

# MX records
dig +short MX domain

# Reverse DNS for a discovered IP (example)
dig +short -x 93.184.216.34
```

Expected notes for report

- Registrar and registration dates (WHOIS).
 - Authoritative name servers and any unusual delegations.
 - Public A records and any shared-host evidence (multiple domains → same IP).
-

Task 2 — Certificate transparency / crt.sh subdomain discovery (10–15 minutes) — background

Why: Public TLS certificates often contain SANs (subjectAltName) listing subdomains; crt.sh and similar CT logs are excellent passive sources for subdomains.

Commands (browser)

Open: <https://crt.sh/?q=%25.domain> and inspect results.

Commands (CLI, parse JSON where available)

```
curl -s "https://crt.sh/?q=%25.domain&output=json" | jq -r '[][.name_value] |
sed 's/\\*\\.//g' | sort -u > crt_sh_subs.txt
cat crt_sh_subs.txt
```

Expected notes

- List of subdomains discovered via certificates.
 - Any unexpected or legacy hosts (e.g., old.domain, dev.domain) to flag.
-

Task 3 — Email & host harvesting with theHarvester (10–15 minutes) — background

Why: theHarvester aggregates search-engine results to find e-mail addresses, hostnames, and sometimes employee names.

Commands

```
# Bing as source (may be limited)
theHarvester -d domain -b bing -l 200 -f theharvester_bing

# Multiple sources (bing, google, linkedin)
theHarvester -d domain -b all -l 200 -f theharvester_all

# View results quickly in terminal

echo "Hosts:"; jq -r '.hosts[]?' theharvester_bing.json || true
echo "Emails:"; jq -r '.emails[]?' theharvester_bing.json || true
```

Expected notes

- Public email addresses and their contexts (admin@, info@).
 - Hostnames and any correlation with certificate or DNS findings.
-

Task 4 — Passive subdomain enumeration (sublist3r, amass) (15 minutes) — background

Why: Subdomain enumeration helps map the attack surface — passive modes query public sources without interacting with hosts.

Commands

```
# sublist3r
sublist3r -d domain -o sublist3r_domain.txt
cat sublist3r_domain.txt | sort -u
```

```
# amass passive
amass enum -passive -d domain -o amass_passive_domain.txt
cat amass_passive_domain.txt | sort -u > combined_subs.txt
```

Expected notes

- Consolidate subdomains from both tools and crt.sh.
 - Note any overlaps and any unique finds per tool.
-

Task 5 — Google dorking (browser) (15 minutes) — background

Why: Carefully-crafted Google queries can reveal exposed admin pages, config files, backups, or other sensitive content indexed by search engines. Only query authorized/demo domains.

Example dorks (replace domain):

```
site:domain inurl:admin
site:domain filetype:sql OR filetype:bak OR filetype:env
site:domain inurl:wp-admin OR inurl:wp-login.php
site:domain "phpinfo()" OR intitle:"phpinfo"
site:domain "password" OR "credentials"
```

Notes

- For each interesting hit record the URL and short reason why it's interesting (do not download content).
 - Try at least three different dorks and record results.
-

Task 6 — Shodan dorking (CLI & browser) (10–15 minutes) — background

Why: Shodan indexes banners collected by Internet-wide crawlers. Use it to find publicly visible devices or services associated with a hostname/IP.

Setup

```
shodan init YOUR_SHODAN_KEY
```

Commands

```
# Search by hostname
shodan search --fields ip_str,port,hostnames "hostname:domain"

# Search for common web-server banners mentioning the domain
shodan search --fields ip_str,port,http.title "http.html:domain"
```

```
# Inspect a specific host (passive info Shodan has)
shodan host 93.184.216.34
```

Notes

- Free Shodan accounts have rate limits: be selective.
 - Record banner snippets, ports, and any services of interest.
-

Task 7 — recon-ng: automated passive collection (detailed task) (25–30 minutes) — background

Why: recon-ng is a modular, database-driven framework that automates many OSINT tasks and makes it easy to repeat collections, correlate results, and export structured reports (CSV/JSON). This task is included to teach structured OSINT workflows.

Start recon-ng

```
recon-ng
```

You will be at the `recon-ng >` prompt.

Step A — Create workspace

```
workspaces create nes555_domain
workspaces list
workspaces select nes555_domain
```

Background: Workspaces isolate projects so multiple target investigations don't mix.

Step B — Add the domain as a target

```
keys add SHODAN_API YOUR_SHODAN_KEY      # optional, for shodan modules
keys add CRITEO_API <if available>        # optional
show domains
```

Step C — Load and run passive modules (examples)

```
marketplace search bing
marketplace install recon/domains-hosts/bing_domain_web

modules search recon/domains-hosts
modules load recon/domains-hosts/bing_domain_web
options set SOURCE domain
run
# check results
show hosts
```

Repeat with other passive modules:

- `recon/domains-hosts/google_site_web` (if allowed)
- `recon/domains-contacts/whois_pocs`
- `recon/domains-hosts/crtsh` (**certificate discovery module**)
- `recon/domains-hosts/shodan_hostname` (requires Shodan key)

Step D — Import/aggregate results

```
show hosts
show contacts
```

Step E — Export findings

```
export csv /home/kali/nes555/lab2/reconng_hosts.csv hosts
export csv /home/kali/nes555/lab2/reconng_contacts.csv contacts
```

Step F — Exit

```
exit
```

Expected notes

- recon-ng will produce structured lists of hosts and contacts.
- Compare recon-ng outputs with theharvester/amass/sublist3r/crt.sh results — recon-ng often finds additional correlations.

Wrap-up questions

1. Which passive source (crt.sh, amass, theHarvester, recon-ng) found the most unique subdomains for your assigned domain and why do you think that is?
2. Did Google dorking reveal anything that other passive tools missed? Give one concrete example and why it is important.
3. How would you responsibly disclose an exposed admin panel or credential leak found during passive reconnaissance?